implemented in libraries of such languages as R or Python are perfectly optimized for file processing.

Files, in the case of smaller sets, can represent the whole dataset. And for large amounts of data, in most cases, a subset of the data will first be selected, through an appropriate query or configuration of various filters and then exported to a file.

(REST) API

Another type of port is the REST API. We use HTTP REST here as an example of the protocol that is most popular at the time of writing this book. Nevertheless, we mean here a type of interface that is mainly intended for consumption by other systems.

This type of port is mainly suitable for working on small subsets of data, because it is usually inefficient for large collections. If we add GraphQL support to this type of port, we get the ability to perform complex and customized queries on the data. And with the addition of HATEOAS, you can gain additional capabilities for automating data browsing using ML/AI.

NOTE *GraphQL* is an open source data query and manipulation language for APIs, and a run time for fulfilling queries with existing data. *HATEOAS* is a part of the REST specification stating that in addition to the result data, responses should include dynamic metadata allowing clients to browse and interpret the results in a hypermedia way (like links in classical web pages).

So this port is great for automating data processing and for relatively low-intensity analytics that do not require large amounts of data.

Streams

By *streams*, we mean messaging-based systems that store information as single messages (e.g., message queues). This type of port is not very popular in the world of data yet, especially as a form of sharing data, because tools for its analytical processing are still under development. Messaging systems are great for distributed processing of large amounts of data, especially when we want to process data in near real time. An example is systems that continuously analyze stock-price changes and generate recommendations for